

IntDiabetes(Three Ensemble Models)

NotebookDataLogsComments (70)

129

Copy & Edit114

In [31]:

interpret_model(catboost)



In [32]:

interpret_model(rf)

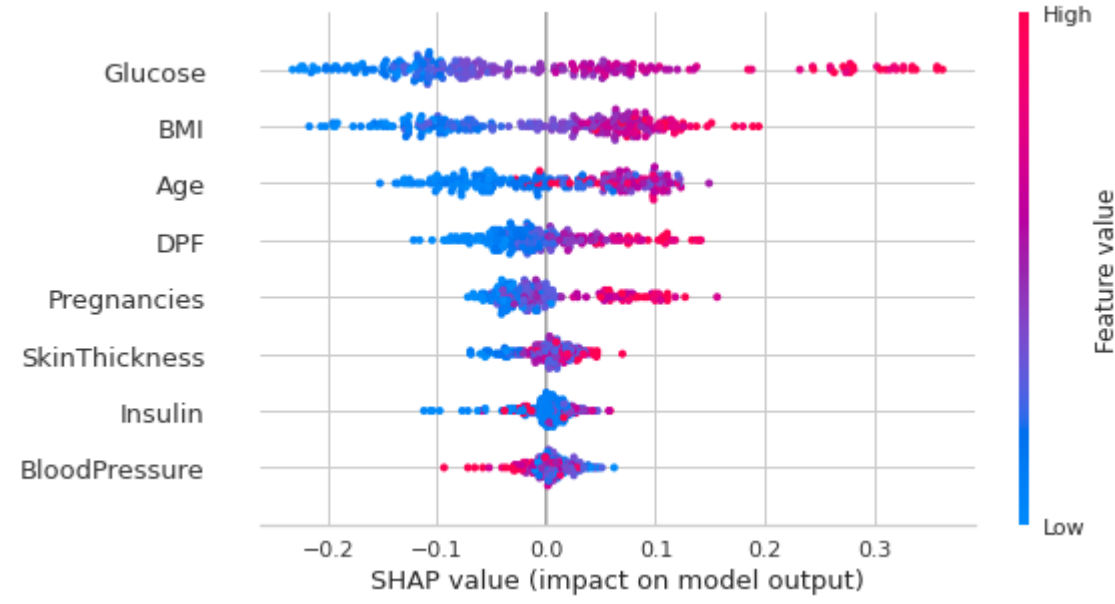


Table of Contents

Introduction

Load Libraries

Defining Utility functions

Reading and Checking data

EDA

Checking and Removing Outliers

Scaling

Checking correlation between...

Checking features before...

Visualizing Training Dataset after

Ensemble

Tuning Hyperparameters

Stacking

Soft Voting

Hard Voting

Predicting with the test dataset

Calibrating the final model

Finalizing the last model

Conclusion

Observation:

- As expected, glucose is used as the most important feature.
- SkinThickness and BloodPressure have the low importance.

Let's compare the feature importance of the above two models. A comparison was made with SHAP values, and the feature importance of each model is slightly different. The diversity of each model seems to be stable and improve performance while compensating for each other's weaknesses.

If you are more interested in feature importance, please refer to the notebook below.

Notebook (<https://www.kaggle.com/ohseokkim/interpreting-models-by-feature-importance>)

Tuning Hyperparameters

This function tunes the hyperparameters of a given estimator. The output of this function is a score grid with CV scores by fold of the best selected model based on optimize parameter.

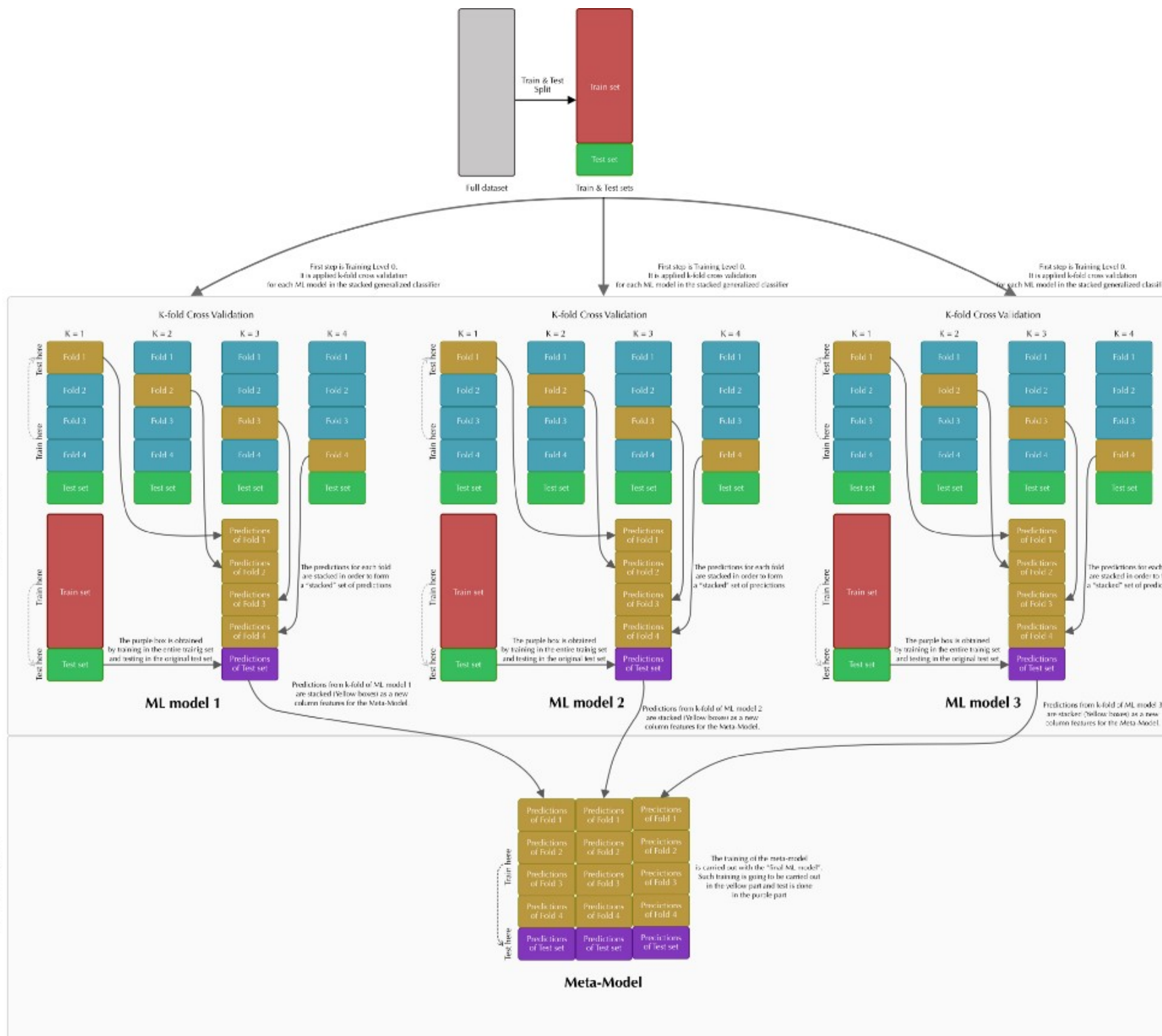
In [33]:

tuned_catboost = tune_model(catboost, optimize = 'AUC')
tuned_rf = tune_model(rf, optimize = 'AUC')
tuned_lr = tune_model(lr, optimize = 'AUC')
tuned_lda = tune_model(lda, optimize = 'AUC')
tuned_gbc = tune_model(gbc, optimize = 'AUC')

Show hidden output

Top 1, Top 2, Top 3 ,Top 4, and Top 5 models were tuned with different feature importance and decision boundary. Also, there is a big difference in feature importance from the catboost classifier.

Stacking



In [34]:

stack_model = stack_models(estimator_list = top5, meta_model = top5[0],optimize = 'AUC')

	Accuracy	AUC	Recall	Prec.	F1	Kappa	MCC
0	0.7222	0.7789	0.3684	0.7000	0.4828	0.3170	0.3476
1	0.7593	0.8722	0.5263	0.7143	0.6061	0.4384	0.4490
2	0.7407	0.8135	0.5789	0.6471	0.6111	0.4176	0.4190
3	0.6481	0.6647	0.2632	0.5000	0.3448	0.1349	0.1479
4	0.7593	0.7940	0.5789	0.6875	0.6286	0.4524	0.4561
5	0.7593	0.8737	0.6316	0.6667	0.6486	0.4658	0.4661
6	0.7778	0.9053	0.5789	0.7333	0.6471	0.4882	0.4954
7	0.7547	0.8576	0.6316	0.6667	0.6486	0.4605	0.4609
8	0.8679	0.9303	0.6842	0.9286	0.7879	0.6952	0.7122
9	0.8302	0.8854	0.5789	0.9167	0.7097	0.5981	0.6297
Mean	0.7619	0.8376	0.5421	0.7161	0.6115	0.4468	0.4584
SD	0.0558	0.0736	0.1224	0.1196	0.1152	0.1421	0.1434

In [35]:

plt.figure(figsize=(8, 8))
plot_model(stack_model, plot='boundary')

In the case of a stacking model, in some cases overfitting and in some cases underfitting.

If you are interested in under/overfitting, please refer to the notebook below.

Notebook (<https://www.kaggle.com/ohseokkim/overfitting-and-underfitting-eda>)

Let's compare the other model's boundary.

In [36]:

plt.figure(figsize=(8, 8))
plot_model(stack_model, plot = 'auc')